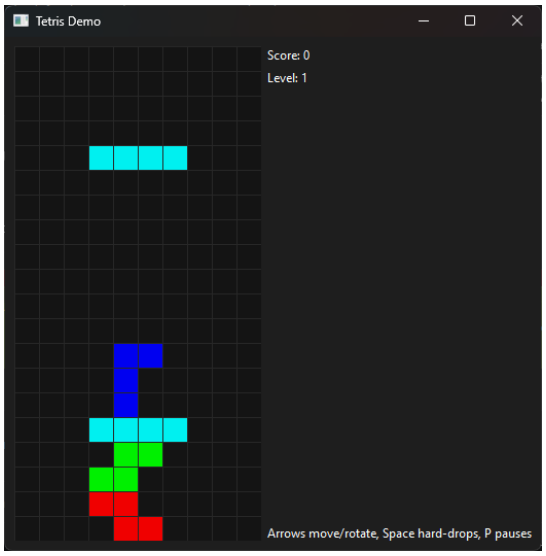
How We Build Software Here

A plain-language walkthrough, using the Tetris demo project as the real example

What We Built



- A small, single-player Tetris game — not the point of the exercise. The game is just a real, working example to walk through the process on.
- What actually matters here is everything that happened around the code: how the project was set up, reviewed, and shipped.

1. Starting a New Project

- Every project starts as an empty folder on GitHub — nothing exists yet, not even a "main" version to compare changes against.
- The very first save (called a "commit") goes directly onto the main version, just to get something there to build on.
- After that one exception, every future change goes through a review process before it's allowed to join the main version.

2. Starting from a Shared Rulebook

- New projects don't start from a blank page — they copy a shared template: a rulebook for how the team works, plus a set of automated checks.
- Copying it isn't blind trust. Each project adapts the template to its own needs.

- We even caught a mistake in the template itself: an old habit (a separate "findings log" file) the team had already replaced with something better on other projects, just not yet updated in the shared copy.
- Lesson: a shared rulebook is a starting point, not gospel. Check it against how things are actually done today, and fix it when it's fallen behind.

3. Building Safely, on a Branch

- All real work happens on a "branch" — an independent line of development, not the project's main version.
- Nothing on that branch affects the real version until it's explicitly approved and merged in.
- Before asking anyone to look at it, the work is tested locally first, using the exact same checks a robot reviewer will run later.

4. Asking for Review

- A "Pull Request" is a formal request: please review this change before it becomes permanent.
- Four automated checks have to pass: a style check, a security scan, the automated tests, and a basic "does it actually run" check.
- An AI reviewer reads every change too, and leaves comments just like a human reviewer would.

5. The Review Loop

- The first automated review found 7 issues — including one real bug: keyboard controls silently did nothing until you clicked the game window.
- Every issue gets fixed, or gets a stated reason for not fixing it. Nothing is silently ignored.
- A second review caught an outdated instruction left over from the shared rulebook.
- A third review caught a genuine crash: following the setup instructions exactly as written would have failed with an error.
- Each fix gets written down in a running notes file, so the same mistake doesn't get made twice on a future project.

Important: Quality Control Takes Time

- Writing the actual game code took well under an hour.
- Getting it fully reviewed and ready to merge took considerably longer — three rounds of review, each with a built-in wait (the AI reviewer's free plan allows one full review per hour).

- This is normal, not a failure. Every single round caught something real.
- Don't measure a task by "time to write the code" alone. Reviewing and fixing is just as much a part of the work, and it can take longer than the writing itself.

6. Merging — and Keeping the Trail

- Once everything passes, the side copy is merged into the real version.
- The side-copy branch is deliberately not deleted afterward.
- Keeping it costs almost nothing, and it leaves a clear trail of exactly what changed and when, for later reference.

7. Marking a Version

- Once a milestone is reached, it gets a version number — like v0.1.0 — attached to that exact point in the project's history.
- This project's numbering: the first number is for breaking changes, the second for new features, the third for small fixes.
- No automatic publishing happens when a version is marked here. It's purely a label: "this is what the project looked like at this point."

This Document Followed the Same Process

- Even this write-up went through a side branch, a review, and a merge — no shortcut for "it's just documentation."

Glossary

- Repo — the project's whole folder of files plus its complete history.
- Branch — an independent line of development, used to make one set of changes without touching the main version until it's ready.
- Commit — a saved snapshot of a set of changes, with a note on what and why.
- Pull Request (PR) — a request to review and merge one branch's changes into another.
- Merge — folding an approved branch's changes into the main version for good.
- Tag — a permanent label marking one exact point in the project's history.
- CI (Continuous Integration) — automated checks that run on every proposed change.
- Linter / security scanner — tools that check code for style issues or risky patterns without running it.

- CodeRabbit — the AI tool that automatically reviews every change and leaves comments, similar to a human reviewer.
- Rate limit — a cap on how often a service can be used in a stretch of time.